

2026-07-01-webauthn-passkey-login

WebAuthn / passkey login for the HelixCode auth gate (design spec — operator review)

Revision: 1 **Last modified:** 2026-07-01T19:08:56Z **Status:** DESIGN — for operator review. NOT implemented. No code/tests changed by this document. **Authority:** Research recommendation #1 in docs/research/auth_modernization_20260701/FINDINGS.md §1.1 / "Top recommendations #1". **Builds on (unchanged, still authoritative):** docs/superpowers/specs/2026-07-01-auth-pivot-ssh-key.md (the SSH-key challenge-response gate) and docs/superpowers/specs/2026-07-01-real-account-code-server-design.md (host-native code-server behind Caddy forward_auth). **Anti-bluff posture:** every "works like X" claim about the WebAuthn library / browser API is cited in "Sources verified" (§11.4.99). Facts not pinned this pass are marked OPEN: in §10, never asserted (§11.4.6). This is a design artefact — it recommends, it does not implement, and it does not substitute runtime validation of the chosen path (§11.4.108 / §11.4.40).

1. Goal

Add **WebAuthn / passkey login as the primary daily UX** for the single real account, while keeping the existing **SSH-key challenge-response as the RECOVERY / bootstrap path**. Concretely:

- **Phishing-resistant, one-gesture login.** Replace "run ssh-keygen -Y sign, copy the armored block, paste it" with a single navigator.credentials.get() tap. The private key never leaves the authenticator; the login proves possession of a key bound to the deployment origin (RP ID), so a credential registered for

our FQDN cannot be replayed against a look-alike site (§1.1 of FINDINGS).

- **Same hardware the user already trusts.** A FIDO2 YubiKey can hold BOTH an sk-ssh-ed25519 SSH key AND a WebAuthn credential, so this is additive — the user taps the same key, the browser handles the challenge instead of a terminal + clipboard (FINDINGS §2).
- **No new identity provider, host-native.** One stored credential for one account; no user directory, no IdP, no session broker, no external dependency. It slots directly into the Gin gate we already own (services/auth_gate/).
- **Zero change to the Caddy contract.** On success the gate issues the SAME __Host-helix_session cookie the SSH-key path issues today; Caddy forward_auth still keys on 2xx from GET / auth — see §4.
- **Fail-closed, degrade cleanly.** WebAuthn unavailable (no JS / no API / disabled by config / user cancels) = the page still offers the SSH-key form. No new stored secret (only a PUBLIC key), no weakening of CSRF / rate-limit / replay defences.

Non-goals (explicitly out of scope for this spec): replacing the bespoke gate with Authelia (FINDINGS §1.5 — a separate decision); the --auth none localhost-socket hardening (FINDINGS §3 — separate work item); WireGuard/Headscale network-layer access (FINDINGS §3 alternative). This spec is ONLY the WebAuthn/passkey addition to the existing gate.

2. Library

github.com/go-webauthn/webauthn — the maintained, FIDO2-conformance-tested Go relying-party backend (the successor to the superseded [duo-labs/webauthn](https://github.com/duo-labs/webauthn)), storage-agnostic and framework-agnostic, a natural fit for Gin (FINDINGS §1.1).

Licence — the research's flagged VERIFY item, now resolved this pass (§11.4.99): the repository LICENSE is **BSD-3-Clause** ("Copyright (c) 2025 github.com/go-webauthn/webauthn authors"), current module version **v0.17.4** (published 2026-05-22). BSD-3-Clause is permissive and compatible with a host-native self-hosted service. **STILL REQUIRED before an implementation commit (§10 OPEN-1):** confirm the SPDX of the pinned version's LICENSE in-tree AND the licences of the transitive deps it pulls (github.com/fxamacker/cbor/v2, github.com/go-webauthn/x, github.com/google/uuid) — a permissive parent with a copyleft transitive dep would still bind us.

Current public API (verified against pkg.go.dev this pass — pin + re-verify at implementation time, §11.4.99):

- `webauthn.New(config *Config) (*WebAuthn, error)` — construct the relying party from `RPDisplayName` / `RPID` / `RPOrigins`.
- Registration: `BeginRegistration(user, ...opt) (*protocol.CredentialCreation, *SessionData, error) → FinishRegistration(user, session, *http.Request) (*Credential, error)`.
- Assertion (server-side `allowCredentials`, non-discoverable): `BeginLogin(user, ...opt) (*protocol.CredentialAssertion, *SessionData, error) → FinishLogin(user, session, *http.Request) (*Credential, error)`.
- Discoverable/passkey variant (usernameless): `BeginDiscoverableLogin(...) → FinishPasskeyLogin(handler, session, *http.Request)`.
- User interface we implement for the single account:
`WebAuthnID() []byte, WebAuthnName() string,`
`WebAuthnDisplayName() string, WebAuthnCredentials() []Credential.`
- `SessionData` is the state carried between `Begin` and `Finish`; the maintainers explicitly recommend persisting it in an **opaque session cookie** — which matches our existing HMAC-signed-token pattern exactly (§3.3).

Consumed as a vendored Go module (§11.4.74 — reuse an existing library, do not reimplement WebAuthn crypto), NOT a git submodule and NOT a new remote. It composes with the `go.mod` at `services/auth_gate/`.

3. Flow

Two ceremonies: a one-time **registration bootstrap** (operator-gated, stores ONE credential) and the daily **assertion login**. Both reuse the gate's existing cookie/session/CSRF/rate-limit/fail-closed machinery.

3.1 Credential choice: server-side (non-discoverable) credential

We use a **non-discoverable, server-side credential** (we always know the single credential ID and place it in `allowCredentials` at login time), NOT a resident/discoverable "usernameless" passkey. Rationale for our single-account model:

- We already know exactly which account is logging in — there is only one — so usernameless discovery buys nothing.

- It avoids consuming a **resident-key slot** on the hardware key (YubiKey 5 holds a limited number of discoverable credentials; non-discoverable credentials are unlimited).
- The UX is still one-tap: the browser prompts for the specific credential named in `allowCredentials`.

Terminology honesty (§11.4.6): a non-discoverable credential is a "FIDO2 device-bound credential", strictly a *passkey superset* term — the user-facing "Sign in with passkey" button is accurate for the gesture, and §10 OPEN-4 records the discoverable-vs-non-discoverable choice as an operator decision (discoverable is a one-line `RequireResidentKey/ResidentKeyRequirement` change if the operator prefers true usernameless).

3.2 Registration bootstrap (one-time, operator-gated)

Trust anchor problem: the single largest WebAuthn threat here is an attacker enrolling *their own* authenticator over the wire (self-enrollment = full account takeover). Registration **MUST** therefore be gated so only the real host operator can do it.

Origin subtlety (load-bearing, §10 OPEN-2):

`navigator.credentials.create()` binds the new credential to the page's **RP ID**, and the browser only permits an RP ID that is a registrable suffix of the *page origin*. So the registration page **MUST** be served from the real deployment origin `https://<FQDN>` (going through Caddy), **NOT** from `localhost` — a credential created on a `localhost` origin would carry RP ID `localhost` and be unusable at the real login. This rules out a pure offline `localhost` CLI enroll unless `/etc/hosts` is spoofed.

Chosen bootstrap model (defence in depth — all three required):

1. **A valid SSH-key session.** The operator first logs in via the existing SSH-key recovery path (§3 of the auth-pivot spec) and holds a valid `__Host-helix_session` cookie. The registration endpoints refuse an unauthenticated caller. (The SSH-key path is the trust anchor for enrolling the first passkey — you must already prove key possession.)
2. **A sudo/operator-minted single-use bootstrap token.** A new CLI subcommand `auth_gate webauthn-bootstrap` (run locally as the host account — the same trust level as being able to read `~/.ssh/authorized_keys`) mints a short-TTL, single-use token (HMAC-signed with the existing cookie secret, domain-separated) and writes it `0600` to `~/.config/helixcode/webauthn_bootstrap_token` (and echoes it to the operator's terminal). The registration finish endpoint requires this token and burns it (reuse `ReplayGuard`).

3. **Refuse-if-a-credential-already-exists.** If the credential file is present, registration is denied unless the operator passes an explicit `--force` to the CLI, which first backs up the existing credential (§9.2 hardlinked/copy backup) before allowing a replacement. This blocks a silent second-credential enrollment.

Endpoints (mounted on the gate, served over the FQDN so RP ID is correct):

- `POST /auth/webauthn/register/begin` — requires (session + valid bootstrap token). Calls `w.BeginRegistration(user, opts...)` with `AuthenticatorSelection{ UserVerification: required, ResidentKey: discouraged }` + `Attestation: none` (we do not need attestation for a self-enrolled single user; requiring it adds a privacy/complexity cost with no benefit here — §10 OPEN-5). Returns the `CredentialCreation` JSON; stores the `SessionData` in a short-TTL signed cookie (§3.3).
- `POST /auth/webauthn/register/finish` — requires (session + the SAME bootstrap token, now burned). Calls `w.FinishRegistration(user, sessionData, r)`; on success persists the returned `Credential` to the credential file (§5) and clears the bootstrap token. Fail-closed on any error (no partial write — write to a temp file in `~/.config/helixcode/` then atomic rename).

The registration UI is a minimal page (served only to an authenticated session with a valid token) with a single "Register this passkey" button driving `navigator.credentials.create()`. It is NEVER part of the normal `/login` page.

3.3 Carrying SessionData between begin and finish (stateless, matches existing architecture)

The gate is deliberately stateless (HMAC-signed tokens, no server session store). We keep that: the WebAuthn `SessionData` (challenge + allowed credential IDs + UV requirement + expiry) is serialized and carried in a **short-TTL, HMAC-signed, single-use __Host-cookie** — call it `__Host-helix_wa` — minted exactly like the existing challenge token (challenge.go's `ChallengeCodec` pattern, domain-separated HMAC over the payload), and its embedded WebAuthn challenge is registered in the existing `ReplayGuard` so a captured begin/finish pair cannot be replayed. This reuses machinery we already trust and keeps the service stateless (no new store, no §11.4.111 index-binding risk).

3.4 Assertion login (daily path)

1. `GET /login` renders the page (§6) with a primary **"Sign in with passkey"** button AND the existing SSH-key form (recovery, behind a "Use SSH key instead" disclosure). The passkey button is revealed by progressive enhancement only when

window.PublicKeyCredential exists (§6, mirrors today's login_enhance.js pattern).

2. POST /auth/webauthn/assert/begin — passes through the SAME middleware chain as POST /login today: mwBodyLimit → mwRateLimit (ALLOW gate, checks-not-consumes) → mwCSRF (double-submit). Calls w.BeginLogin(user) (non-discoverable → allowCredentials = our one credential). Returns the CredentialAssertion request-options JSON and sets the __Host-helix_wa signed session cookie (§3.3).
3. Client runs navigator.credentials.get(options) — the authenticator does user-verification (PIN/biometric/touch) and signs the challenge.
4. POST /auth/webauthn/assert/finish — same middleware chain (body-limit, rate ALLOW gate, CSRF). Loads + verifies the __Host-helix_wa cookie (HMAC valid, unexpired, challenge not already claimed in ReplayGuard); calls w.FinishLogin(user, sessionData, r). On success:
 - **verify the authenticator sign-count advanced** (clone-detection; go-webauthn surfaces this — a non-increasing counter from a counter-supporting authenticator ⇒ reject as a possible clone);
 - **persist the new sign-count** to the credential file (atomic rename);
 - clear throttle state (limiter.RecordSuccess), mint the SAME session cookie via s.codec.Sign(s.cfg.Account, now) (identical to the SSH-key success path in server.go:handleLoginPost), clear __Host-helix_wa, redirect to /.
 - On failure: record a rate-limit failure (a genuine assertion failure is budget-consuming, exactly like a genuine SSH signature failure today), generic 401, no cookie.

Concurrency note (§A4-3 reuse): WebAuthn assertion verification is pure-Go crypto (no ssh-keygen exec), so it does NOT need the verifySem process-spawn ceiling. It IS still bounded by the rate limiter + the body-size cap; add a modest independent CPU-concurrency ceiling for the finish handler if load-testing shows it warranted (cheaper than the exec path, so the existing default is likely ample — confirm under §7 stress test).

4. Caddy forward_auth contract — UNCHANGED

GET /auth (server.go:handleAuth) is untouched: a valid __Host-helix_session cookie ⇒ 200 + X-Helix-User: <account>; an unauthenticated browser navigation ⇒ 303 /login; everything else ⇒ 401. Because the WebAuthn path issues the **same** session cookie the SSH-key path issues, Caddy still only ever sees "2xx = allow"

on /auth. **No Caddyfile change is required** (FINDINGS §1.6 / §1.1). The Caddyfile continues to forward_auth the gate and copy_headers X-Helix-User.

5. Config + credential storage

5.1 New config fields (env-driven, decoupled per §11.4.28 — same style as config.go)

Env var	Meaning	Default
HELIX_WEBAUTHN_ENABLED	Master switch for the WebAuthn path. When false, the gate is pure SSH-key (today's behaviour) and no passkey button renders.	false (opt-in)
HELIX_WEBAUTHN_RPID	Relying-Party ID = the deployment FQDN (e.g. helixcode.example.com). REQUIRED when enabled; empty-while-enabled = fail closed at load (§11.4.6).	(none)
HELIX_WEBAUTHN_RPORIGINS	Comma-separated allowed origins.	https://<RPID>
HELIX_WEBAUTHN_RP_DISPLAY_NAME	Human label shown in the authenticator prompt.	HelixCode
HELIX_WEBAUTHN_USER_VERIFICATION	required preferred.	required
HELIX_WEBAUTHN_CREDENTIAL_PATH	Path to the single-credential JSON file.	~/.config/helixcode/webauthn_credential.js

Mode handling: rather than overload the existing HELIX_AUTH_MODE (which today rejects anything but sshkey, config.go:LoadConfig), keep Mode=sshkey as the base and treat WebAuthn as an **additive capability** gated by HELIX_WEBAUTHN_ENABLED. SSH-key is ALWAYS available as recovery; WebAuthn is layered on. (Alternative — a sshkey+webauthn mode value — is recorded as §10 OPEN-6; the additive-flag approach is preferred because it keeps SSH-key recovery unconditional.)

Fail-closed at load (§11.4.6, mirrors existing LoadConfig validation): ENABLED && RPID==" → error; malformed origin list → error; UV value not in {required,preferred} → error; credential path un-writable parent → error. No silent default substitution.

5.2 Credential file format + secrecy (§11.4.10 / §11.4.30)

A single JSON object holding ONE credential — all **PUBLIC** material (no private key ever touches this service, exactly as the SSH-key path only reads `authorized_keys`):

```
{
  "user_handle": "<base64url 16 random bytes, minted once at
bootstrap>",
  "credential_id": "<base64url>",
  "public_key": "<base64url COSE key>",
  "aaguid": "<base64url>",
  "sign_count": <uint32>,          // updated on every
successful assertion
  "transports": ["usb","nfc"],
  "created_at": "<RFC3339>"
}
```

- Stored 0600 in `~/.config/helixcode/` (parent 0700) — same discipline as the cookie secret (`cookie.go:loadOrCreateSecret`).
 - **Git-ignored (§11.4.30)** — add `webauthn_credential.json` + `webauthn_bootstrap_token` to the gate's `.gitignore`; they must NEVER be tracked. It holds no secret (only a public key), so a leak is not a credential compromise — but it identifies the enrolled authenticator, so keep it out of git and out of the CodeGraph index (§11.4.78) regardless.
 - `user_handle` is a **random** value (not the account name) to avoid embedding identity in the credential (privacy; also correct WebAuthn practice — the user handle is not meant to be PII).
 - **Regeneration mechanism (§11.4.77):** the file is NOT re-derivable — it is genuine enrolled state. Its "regeneration" is: re-run the registration bootstrap (§3.2). Document that in `.gitignore-meta/` so a fresh clone knows the file is operator-enrolled, not build output.
-

6. Endpoints + login-page changes

6.1 Endpoint summary

Method + path	Auth required	Purpose	Change
GET /healthz	none	liveness	unchanged
GET /auth	session cookie	Caddy forward-auth check	unchanged
GET /login	none	login page	modified — add passkey button + progressive enhancement
POST /login	CSRF + rate	SSH-key challenge-response (recovery)	unchanged
POST /logout	—	clear session	unchanged
POST /auth/webauthn/assert/begin	CSRF + rate	issue assertion options + <code>__Host-helix_wa</code>	new
POST /auth/webauthn/assert/finish	CSRF + rate	verify assertion → session cookie	new
POST /auth/webauthn/register/begin	session + bootstrap token	issue creation options	new (gated)
POST /auth/webauthn/register/finish	session + bootstrap token	persist credential	new (gated)

All four new endpoints go through the SAME `mwBodyLimit` / `mwRateLimit` / `mwCSRF` middleware where they take a body (the register endpoints additionally require the session + bootstrap-token guard). The handlers stay framework-agnostic behind `wrapHTTP`, matching the existing pattern.

6.2 Login-page changes (loginPageTemplate + a new enhancement module)

- Add a primary **"Sign in with passkey"** <button> above the SSH-key section, laid out in its own row (no overlap / no label-over-label per §11.4.162), with a visible SVG key icon and the label text "Sign in with passkey".
 - Wrap the existing SSH-key form in a <details>/"Use SSH key instead" disclosure so the passkey path is visually primary and the recovery path is one click away.
 - A new client module assets/webauthn_login.js (embedded verbatim, same trust model as login_enhance.js — developer-authored, carries NO request-derived data, §11.4.10) that: reveals the passkey button only when `window.PublicKeyCredential` is present; on click, `fetch('/auth/webauthn/assert/begin')` → `navigator.credentials.get()` → `fetch('/auth/webauthn/assert/finish')` → follow the redirect; on cancel/error, reveals the SSH-key form and shows a generic status (never leaks why).
 - With JavaScript OFF, or WebAuthn absent, the page renders the SSH-key form exactly as today (progressive enhancement — the passkey button simply never appears).
-

7. Security analysis + threat model

Composes the existing gate's hardening lessons (§A1–§A5 in the code comments) and the anti-bluff covenant.

Threat	Mitigation
Phishing / origin spoofing	WebAuthn binds the credential to RPID; RPOrigin pinned to exactly <code>https://<FQDN></code> . A credential minted for our origin cannot assert against a look-alike (FINDINGS §1.1). Strictly stronger than the SSH-key paste path, which is not origin-bound.
Self-enrollment over the wire (account takeover)	Registration requires (valid SSH-key session) AND (sudo-minted single-use bootstrap token) AND (refuse-if-credential-exists). An attacker on the network has none of the three. §3.2.
Assertion replay	WebAuthn challenge is single-use via ReplayGuard + the <code>__Host-helix_wa</code> cookie is short-TTL HMAC-signed. Mirrors the existing SSH-challenge replay guard.
Cloned authenticator	Sign-count monotonicity checked on every finish; a non-increasing counter ⇒ reject. New count persisted atomically.

Threat	Mitigation
CSRF on begin/finish	Existing double-submit (<code>__Host-helix_csrf</code> vs hidden field, constant-time <code>hmac.Equal</code>) applies to all four new endpoints — defence in depth on top of WebAuthn's own origin binding.
Rate-limit DoS / logout of the sole user	Reuse the existing ALLOW-gate limiter: only a GENUINE assertion failure consumes budget (as with a genuine signature failure today, <code>server.go:handleLoginPost</code> step 7). A no-CSRF / forged- <code>__Host-helix_wa</code> / expired-challenge POST never burns the victim's budget.
Stored-secret exfiltration	None added — the credential file is a PUBLIC key. No password, no private key, no shared secret introduced.
Credential file tamper / swap	0600 in a 0700 dir; atomic-rename writes; git-ignored + index-excluded. A tampered public key simply fails the assertion (fail-closed).
Downgrade to the SSH-key path	The fallback path is itself strong (key-possession, no stored secret) — so "forcing" fallback does not weaken security to a password. Honest note: the SSH-key path is NOT origin-bound the way WebAuthn is, so an operator who wants strict phishing-resistance everywhere may later gate the recovery path behind an additional confirmation (§10 OPEN-7).
Lost / broken authenticator (availability)	The SSH-key challenge-response is ALWAYS available as recovery — this is the deliberate second factor for the single account (FINDINGS §1.1 "account-recovery is the hard part").
Fail-closed everywhere	Config error, CSPRNG failure minting <code>__Host-helix_wa</code> or the session cookie, missing/corrupt credential file, WebAuthn verify error ⇒ deny, generic 401/503, no cookie. Matches the existing gate's posture.
Honest boundary (§11.4.6): WebAuthn removes the phishing + replay + stored-secret classes for the daily path; it does NOT by itself close the <code>--auth none</code> localhost residual (FINDINGS §3, separate item) nor change the SSH-server story (FINDINGS §1.3). Those remain tracked separately.	

8. Test plan (four-layer, §11.4.4(b) / anti-bluff)

Unit (mocks permitted only here, §11.4.27):

- Credential file load/save round-trip (byte-stable), 0600 perms asserted, atomic-rename-on-write, corrupt-file ⇒ fail-closed.
- Config load: ENABLED && RPID="" ⇒ error; bad origins ⇒ error; bad UV value ⇒ error (table-driven, mirrors config_test.go).
- `__Host-helix_wa` session-data codec: sign/verify round-trip, tamper ⇒ reject, expiry ⇒ reject, single-use via ReplayGuard.
- Bootstrap-token mint/verify: single-use, TTL, wrong-token ⇒ reject.
- FinishLogin outcome mapping: sign-count regression ⇒ reject (clone); wrong-RP-ID assertion ⇒ reject; assertion for an unknown credential ID ⇒ reject.

Integration (real ceremony, NO human, re-runnable per §11.4.98):

- **Primary:** github.com/descope/virtualwebauthn (pure-Go software authenticator, verified this pass) drives the FULL ceremony against the real begin/finish endpoints with no browser: bootstrap register → assert/begin → software get() → assert/finish → assert `__Host-helix_session` issued → GET /auth returns 200. Negatives: replayed challenge ⇒ 401; tampered clientDataJSON ⇒ 401; wrong-origin assertion ⇒ 401; sign-count non-advance ⇒ 401. This is the §11.4.98 fully-autonomous, -count=3-stable layer. (Honest boundary: a software authenticator validates OUR server logic, not a real browser's enforcement.)
- **Browser-realism:** CDP `WebAuthn.addVirtualAuthenticator` (Chromium-only, verified this pass) via Playwright/chromedp — protocol=ctap2, transport=internal, hasUserVerification=true, isUserVerified=true, UI disabled — drives the REAL `navigator.credentials.get()` against the served /login page end-to-end. Covers the browser-side origin/RP-ID enforcement the software authenticator cannot. (WebKit/Firefox lack CDP virtual authenticators — document as a §11.4.3 SKIP for those engines, never a faked pass.)

§11.4.170 host-rendered UI visual proof (device-independent, mandatory for the UI change):

- Render the /login page (with `HELIX_WEBAUTHN_ENABLED=true`) to a PNG **on the host** via Playwright headless — for {light, dark} (the template already sets color-scheme: light dark).
- Dual validation: (i) golden image-diff of the "Sign in with passkey" button + the "Use SSH key instead" disclosure; (ii) an OCR/vision oracle asserting the literal label "Sign in with passkey" is legible, within bounds, and NOT overlapping the

SSH-key section (no label-over-label, no clipping, no collapsed/giant widget — §11.4.162 invariants on real pixels).

- Self-validated golden-good/golden-bad analyzer (§11.4.107(10)): a golden-bad frame with the button clipped/overlapping MUST FAIL, or the analyzer is a bluff gate. Value-equality/token assertions may SUPPLEMENT but NEVER substitute this rendered-pixel proof.

E2E (full journey through Caddy, recorded per §11.4.153/.158/.159):

- Unauth request ⇒ 303 /login ⇒ passkey assert (CDP virtual authenticator) ⇒ __Host-helix_session ⇒ protected route 200 ⇒ POST /logout ⇒ 303 /login.
- SSH-key recovery path still works with WebAuthn enabled (no regression).
- Window-scoped MP4, vision-verified content read (§11.4.159(D)), <project-prefix>-named (§11.4.155).

Meta-test (§1.1 paired mutation): strip the sign-count-regression check ⇒ the clone test must FAIL; strip the RPOrigins pin ⇒ the wrong-origin test must FAIL; strip the bootstrap-token guard ⇒ a self-enroll test must FAIL.

9. Reuse / churn summary

Reused unchanged: cookie HMAC codec + secret (cookie.go), session TTL, ReplayGuard (challenge.go), rate limiter (ratelimit.go), mwBodyLimit/mwRateLimit/mwCSRF middleware, GET /auth, POST /logout, GET /healthz, fail-closed posture, Caddy forward_auth render, systemd --user unit, install flow, TLS. The Verifier SSH-key path (verifier.go) stays as-is (recovery).

New / changed: vendored go-webauthn/webauthn module; a webauthn.User impl for the single account; the credential-file store; the __Host-helix_wa session-data codec; four new endpoints; the webauthn-bootstrap CLI subcommand; loginPageTemplate + assets/webauthn_login.js; new Config fields + LoadConfig validation; .gitignore + .gitignore-meta/ entries; .env.example documentation.

10. Honest open questions / risks (operator decisions before implementation)

- **OPEN-1 — Licence (partially resolved).** go-webauthn/webauthn is BSD-3-Clause at v0.17.4 (verified this pass). STILL

DO before an implementation commit: confirm the pinned version's in-tree LICENSE SPDX AND the transitive-dep licences (fxamacker/cbor/v2, go-webauthn/x, google/uuid) — §11.4.99. This was the research's flagged blocker.

- **OPEN-2 — Bootstrap origin / trust model.** Registration MUST be served from the real FQDN origin (RP-ID binding), so a pure offline localhost CLI enroll does not work. The proposed model is (SSH-key session + sudo-minted single-use token + refuse-if-exists). Operator to confirm this is acceptable vs a spoof-/etc/hosts offline enroll or an SSH-tunnel-with-Host-header approach.
 - **OPEN-3 — Browser + authenticator support.** WebAuthn Level 3 is broadly supported, but confirm the operator's actual browsers AND that their specific YubiKey firmware co-hosts an sk-ssh-ed25519 SSH key and a FIDO2 credential without slot conflict. CDP virtual authenticators are Chromium-only (WebKit/Firefox e2e is a documented SKIP).
 - **OPEN-4 — Discoverable vs non-discoverable credential.** Spec recommends non-discoverable (server-side allowCredentials) to avoid resident-slot consumption; discoverable/usernameless is a one-flag change if the operator prefers true passkey UX. Decide before enrolling.
 - **OPEN-5 — Attestation.** Spec recommends Attestation: none (no benefit for a self-enrolled single user; avoids privacy/complexity cost). Confirm the operator does not want attestation-pinned enrollment.
 - **OPEN-6 — Mode modelling.** Additive HELIX_WEBAUTHN_ENABLED flag (keeps SSH-key recovery unconditional) vs a sshkey+webauthn HELIX_AUTH_MODE value. Spec recommends the additive flag.
 - **OPEN-7 — Single credential vs a backup authenticator.** The operator's instruction is "single credential"; best practice is a second enrolled key for availability. Spec honours "single credential" and relies on the SSH-key path as the backup — confirm that is the intended recovery story, or relax to a small fixed set (≤ 3).
 - **OPEN-8 — API drift.** Verified against go-webauthn v0.17.4 this pass; pin the version and re-verify BeginLogin/FinishLogin/User signatures at implementation time (§11.4.99).
-

Sources verified 2026-07-01

- go-webauthn LICENSE (BSD-3-Clause, "Copyright (c) 2025 ..."): <https://github.com/go-webauthn/webauthn/blob/master/LICENSE> (accessed 2026-07-01)
- go-webauthn API + v0.17.4 (New / BeginRegistration / FinishRegistration / BeginLogin / FinishLogin / BeginDiscoverableLogin / FinishPasskeyLogin / User interface / SessionData opaque-cookie guidance): <https://>

pkg.go.dev/github.com/go-webauthn/webauthn/webauthn
(accessed 2026-07-01)

- descope/virtualwebauthn (pure-Go, no-browser register + login ceremony testing): <https://github.com/descope/virtualwebauthn> and <https://www.descope.com/blog/post/virtual-webauthn> (accessed 2026-07-01)
- Chrome DevTools Protocol WebAuthn domain (addVirtualAuthenticator, ctap2, transport, hasUserVerification, isUserVerified): <https://chromedevtools.github.io/devtools-protocol/tot/WebAuthn/> and <https://developer.chrome.com/docs/devtools/webauthn> (accessed 2026-07-01)
- Passkeys E2E Playwright via WebAuthn virtual authenticator (Chromium-only caveat): <https://www.corbado.com/blog/passkeys-e2e-playwright-testing-webauthn-virtual-authenticator> (accessed 2026-07-01)
- Caddy forward_auth (2xx=allow, copy_headers) + FIDO2/WebAuthn primitives + code-server local-bypass context: inherited from docs/research/auth_modernization_20260701/FINDINGS.md "Sources verified 2026-07-01" (accessed 2026-07-01)

Negative findings / gaps (§11.4.99(B)): the transitive-dependency licences of go-webauthn/webauthn and the operator's specific YubiKey-firmware SSH+FIDO2 slot behaviour were NOT pinned this pass — recorded as OPEN-1 / OPEN-3, to be confirmed against the primary source before any implementation commit. This document is design-only and does not substitute runtime validation (§11.4.108 / §11.4.40).